# Code Explanation

**DLT Pipeline Code Explanation**

This DLT pipeline is designed to process customer and order data. The pipeline consists of multiple tables, starting from raw data ingestion to cleaned data and finally to business value tables.

## Overview of the Code
The code defines a DLT pipeline that ingests raw customer and order data from CSV files, cleans and processes the data, and finally creates business value tables. The pipeline consists of bronze, silver, and gold tables. Bronze tables store raw data, silver tables store cleaned data, and gold tables store business value data.

## Step 1: Defining Bronze Tables for Raw Data Ingestion
The first step is to define the bronze tables that ingest raw customer and order data from CSV files.
The `customer_bronze` and `order_bronze` tables are defined using the `@dlt.table` decorator.

```
@dlt.table(
    name="customer_bronze",
    comment="Raw customer data from CSV"
)
def customer_bronze():
    return spark.read.option("header", "true").option("inferSchema",
 "true")
        .csv("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customer
data")
```

## Step 2: Defining Silver Tables for Cleaned Data
The next step is to define the silver tables that store cleaned customer and order data.
The `customer_silver` table drops null values and duplicates from the `customer_bronze` table, while the `order_silver` table does the same and also calculates the `TotalAmount`.

```
@dlt.table(
    name="customer_silver",
    comment="Cleaned customer data with no nulls or duplicates"
)
def customer_silver():
    return dlt.read("customer_bronze")
        .na.drop()
        .dropDuplicates()

@dlt.table(
    name="order_silver",
    comment="Cleaned order data with TotalAmount calculated"
)
def order_silver():
    return dlt.read("order_bronze")
        .na.drop()
        .dropDuplicates()
        .withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))
```

## Step 3: Defining Gold Tables for Business Value Data
The final step is to define the gold tables that store business value data.
The `ordersummary` table is an SCD Type 2 table that joins customer and order data, while the `customeraggregatespend` table aggregates customer spending by name and date.

```
@dlt.table(
    name="ordersummary",
    comment="SCD Type 2 table with customer and order data",
    table_properties={
        "quality": "gold",
        "delta.enableChangeDataFeed": "true"
    }
)
@dlt.expect_all_or_drop({
    "valid_custid": "CustId IS NOT NULL",
    "valid_orderid": "OrderId IS NOT NULL"
})
def ordersummary():
    # Read the latest silver data
    customer_df = dlt.read("customer_silver")
    order_df = dlt.read("order_silver")
    # ... rest of the code
```

## Step 4: Implementing SCD Type 2 Logic in ordersummary Table
The `ordersummary` table implements SCD Type 2 logic by checking for changes in customer data and updating the existing records accordingly.

```
try:
    existing_data = dlt.read("ordersummary")
    changed_records = new_data.join(
        existing_data.filter(col("IsActive") == True),
        "CustId",
        "inner"
    ).where(
        (new_data["Name"] != existing_data["Name"]) |
        (new_data["EmailId"] != existing_data["EmailId"]) |
        (new_data["Region"] != existing_data["Region"])
    ).select(new_data["CustId"]).distinct()
    # ... rest of the code
```

## Step 5: Aggregating Customer Spending in customeraggregatespend Table
The `customeraggregatespend` table aggregates customer spending by name and date using the `ordersummary` table.

```
@dlt.table(
    name="customeraggregatespend",
    comment="Aggregated customer spending by name and date"
)
def customeraggregatespend():
    ordersummary_df = dlt.read("ordersummary")
    return ordersummary_df.filter(col("IsActive") == True)
        .groupBy("Name", "Date")
```

```
        .agg(spark_sum("TotalAmount").alias("TotalAmount"))
```